Call by Reference

The **call by reference** method of passing arguments to a function copies the reference of an argument into the formal parameter. Inside the function, the reference is used to access the actual argument used in the call. This means that changes made to the parameter affect the passed argument.

To pass the value by reference, argument reference is passed to the functions just like any other value. So accordingly you need to declare the function parameters as reference types as in the below function **Swap()**, which exchanges the values of the two integer variables pointed to by its arguments.

Example:

Program to Swap the values.

```
#include<iostream>
void Swap(int &,int &);
main( )
{
        int num1,num2;
        cout<<"enter the num1 and num2:"<<endl;
        cin>>num1>>num2;
        cout<<"before Swaping "<<endl;
        cout<<"num1 : "<<num1<<" num2 : "<<num2<<endl;
        Swap(num1,num2);
        cout<<"after Swaping "<<endl;
        cout<<"num1 : "<<num1<<" num2 : "<<num2<<endl;
}
```

Vector India

```
void Swap(int &a, int &b)
{
        int temp;
        temp=a;
        a=b;
        b=temp;
}
```

Output:

```
        enter the num1 and num2:
        10
        20
        before Swaping
        num1 : 10   num2 : 20
        after Swaping
        num1 : 10   num2 : 20
```

one of the major disadvantages of pass by value is that all arguments passed by value are copied into the function parameters. When the arguments are large structs or classes, this can take a lot of time. References provide a way to avoid this penalty. When an argument is passed by reference, a reference is created to the actual argument (which takes minimal time) and no copying of values takes place. This allows us to pass large structs and classes with a minimum performance penalty.

Vector India

References allow the function to change the value of the argument, which is undesirable when we want an argument be read-only. If we know that a function should not change the value of an argument, but don't want to pass by value, the best solution is to pass by const reference.

Example :

```
#include<iostream>

using namespace std;


void Print(const int &);


main()

{

        int num=10;

        Print(num);

}


void Print(const int &x)

{

    x++;    // compile time error : a const reference will not allow its value to be changed

}
```

Vector India

Return by reference

Similar to pass by address, values returned by reference must be variables (you should not return a reference to a literal or an expression that resolves to a temporary value, as those will go out of scope at the end of the function and you'll end up returning a dangling reference). When a variable is returned by reference, a reference to the variable is passed back to the caller. The caller can then use this reference to continue modifying the variable, which can be useful at times. Return by reference is also fast, which can be useful when returning structs and classes.

However, just like return by address, you should not return local variables by reference. Consider the following example :

```
int & fun(int data)
{
        int value=x*2;
        return value;   //  return a reference to value
} // value is destroyed here
```

In the above program, the program is returning a reference to a value that will be destroyed when the function returns. This would mean the caller receives a reference to garbage. Fortunately, your compiler will probably give you a warning or error if you try to do this.

Above problem can be solved by making value as static.

```
int & fun(int data)
{
        static int value=x*2;
        return value;   //  return a reference to value
}  // value will not destroyed since it is stored in the datasection.
```

Vector India